**You said:**

# Brick Breaker Game

This project is a terminal-based Brick Breaker (Breakout-style) clone using **Pygame**. It introduces students to interactive game design using object-oriented principles and real-time graphical rendering.

---

## What's Provided

A partially working version of a Brick Breaker game with:

- A player-controlled paddle and a ball that bounces off walls, the paddle, and bricks
- A grid of destructible bricks
- Lives and score display

You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional.

### Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

---

## Getting Started

### Setup

1. Clone the repo or download the project folder.
2. Make sure you have Python 3.10+ installed.
3. Install dependencies:

```
pip install -r requirements.txt
```

4. Run the game:

```
python main.py
```

## Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

### Task 1: Refine Collision Detection

> The ball always bounces vertically off the paddle and bricks, no matter which side it actually hit. Hitting a brick from the left or right should send the ball sideways, but instead it keeps moving in the same horizontal direction, which can look wrong or let it clip past a neighboring brick. Investigate and enhance collision accuracy so the bounce direction matches the side that was actually hit.

### Task 2: Implement Game Over Condition

> Add a screen that displays whether the player won (cleared all the bricks) or lost (ran out of lives), along with the final score, then gracefully waits for input instead of just printing to the console.

### Task 3: Add Replay Option

> After the end screen, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard ball speed/paddle size), or exit.

### Task 4: Add Sound Feedback

> Add basic sound effects for a brick breaking, the ball hitting the paddle or a wall, and the game-over/win moment.

## Expected Behavior

- Smooth paddle movement using `Left` /`Right` or `A` /`D`

- The ball bounces off the side walls, the top wall, the paddle, and bricks
- Hitting a brick destroys it and increases the score
- Letting the ball fall past the paddle costs a life; losing all lives ends the game
- Destroying every brick wins the game

---

## Folder Structure

```
brick-breaker-main/
├── main.py
├── requirements.txt
├── game/
│   ├── game_engine.py
│   ├── paddle.py
│   ├── ball.py
│   └── brick.py
└── README.md
```

---

## Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

this from everything from the start

**ChatGPT said:**

Alright Stitch, before we touch any code, let's treat this like a proper lab submission and break down **exactly what they're expecting**, because this assignment is actually testing both **game programming** and **your ability to use an LLM as a debugging partner**.

# 1. What is Brick Breaker?

If you've ever played Breakout:

🧱 Bricks at the top
🏓 Paddle at the bottom
🟣 Ball bounces around

Goal:

- Ball hits bricks
- Bricks disappear
- Score increases
- Don't let the ball fall below the paddle
- Lose all lives = Game Over
- Break all bricks = Win

---

# 2. What Is Already Given?

They are giving you a **partially working game**.

You are NOT building from scratch.

You are expected to:

1. Read existing code
2. Find bugs
3. Improve functionality
4. Use ChatGPT/Claude during the process

---

# 3. Folder Structure

You'll probably see:

```
brick-breaker-main/
│
├── main.py
│
├── game/
│   ├── game_engine.py
```

```
│   ├── paddle.py
│   ├── ball.py
│   └── brick.py
```

Think of each file as a class.

---

## main.py

Usually starts the game.

Example:

```
from game.game_engine import Game

game = Game()
game.run()
```

This is the entry point.

---

## paddle.py

Controls:

```
class Paddle:
```

Contains:

- Position
- Width
- Height
- Movement

Example:

```
self.rect.x += speed
```

---

## ball.py

Contains:

```
class Ball:
```

Stores:

```
self.x
self.y
self.dx
self.dy
```

where:

```
dx = horizontal velocity
dy = vertical velocity
```

Example:

```
dx = 5
dy = -5
```

means:

```
5 pixels right
5 pixels up
```

every frame.

---

## brick.py

Stores each brick.

Example:

```
class Brick:
```

Contains:

```
x
y
width
height
alive
```

When hit:

```
alive = False
```

or

```
bricks.remove(brick)
```

---

### game_engine.py

The most important file.

Usually contains:

```
while running:
```

Game loop.

---

# 4. Understanding Pygame First

Everything in Pygame runs inside:

```
while running:
```

Example:

```
while running:

    handle_input()

    update_game()

    draw_screen()

    pygame.display.flip()
```

This repeats:

```
60 times per second
```

---

# 5. How Collision Detection Works

The assignment's main bug is here.

Pygame uses rectangles.

Example:

```
ball.rect
brick.rect
```

Check collision:

```
if ball.rect.colliderect(brick.rect):
```

returns:

```
True
```

if they overlap.

---

# Current Broken Behavior

Suppose:

```
 ---> Ball
[Brick]
```

Ball hits brick's side.

Expected:

```
Ball should bounce sideways
```

Meaning:

```
dx = -dx
```

---

But current code probably does:

```
dy = -dy
```

for ALL collisions.

So:

```
Hit top -> dy reversed
Hit side -> dy reversed
Hit corner -> dy reversed
```

which is wrong.

---

# Task 1 Explained Fully

## Current Code Probably Looks Like

```
if ball.rect.colliderect(brick.rect):
    ball.dy *= -1
```

Very common beginner implementation.

---

# Correct Approach

Need to determine WHICH SIDE was hit.

---

Imagine:

```
brick
```



Ball could hit:

```
Top
Bottom
Left
Right
```

---

One way:

```
overlap_left =
    ball.rect.right - brick.rect.left

overlap_right =
    brick.rect.right - ball.rect.left
```

```
overlap_top =
    ball.rect.bottom - brick.rect.top

overlap_bottom =
    brick.rect.bottom - ball.rect.top
```

Smallest overlap usually indicates collision side.

---

Example:

```
min_overlap = min(...)
```

If:

```
min_overlap == overlap_left
```

then:

```
ball.dx = -abs(ball.dx)
```

---

If:

```
min_overlap == overlap_top
```

then:

```
ball.dy *= -1
```

---

This is likely what Task 1 wants.

---

# Task 2 Explained

Current game probably does:

```
if lives == 0:
    print("Game Over")
```

or

```
running = False
```

---

They want:

```
================
YOU WIN
Score: 500
Press R to Replay
Press ESC to Exit
================
```

inside the Pygame window.

---

Need a game state.

Example:

```
PLAYING
WIN
LOSE
```

---

Variable:

```
game_state = "PLAYING"
```

---

When lives become 0:

```
game_state = "LOSE"
```

---

When all bricks destroyed:

```
game_state = "WIN"
```

---

Then draw different screens.

---

Example:

```
font.render(
    "YOU WIN",
    True,
    (255,255,255)
)
```

---

# Task 3 Explained

Replay system.

Current game:

```
Game ends
Program closes
```

Wanted:

```
YOU WIN

1 - Easy
2 - Medium
3 - Hard
ESC - Exit
```

---

Difficulty affects:

### Easy

```
ball_speed = 4
paddle_width = 140
```

---

### Medium

```
ball_speed = 6
paddle_width = 110
```

---

### Hard

```
ball_speed = 8
paddle_width = 80
```

---

Need a reset function.

Example:

```
def reset_game():
```

that:

- recreates bricks
- resets score
- resets lives
- recreates paddle
- recreates ball

---

# Task 4 Explained

Sound effects.

Pygame:

```
pygame.mixer.init()
```

Load:

```
brick_sound =
pygame.mixer.Sound("brick.wav")
```

---

When brick breaks:

```
brick_sound.play()
```

---

When paddle hit:

```
paddle_sound.play()
```

---

When win:

```
win_sound.play()
```

---

When lose:

```
lose_sound.play()
```

---

# What Your Workflow Should Be

### Step 1

Run:

```
python main.py
```

Record:

```
BEFORE video
```

Show:

- weird collision
- no end screen

10 seconds enough.

---

### Step 2

Open files.

Start with:

```
main.py
game_engine.py
```

because that's where almost everything is coordinated.

---

### Step 3

Paste the files here.

Start with:

`main.py`

and then

`game_engine.py`

I'll walk through every line, explain what it does from absolute basics, identify where each task belongs, and help you implement all four tasks one by one so you end up with something that both works and is easy to explain in your lab viva.

**You said:**

i have to clone the repo onto vs code

**ChatGPT said:**

Yup. First let's get the project onto your machine before touching any code.

## Step 1: Get the GitHub repo link

You should have either:

`https://github.com/username/brick-breaker-main.git`

or a ZIP download link.

Tell me which one you have.

---

## Step 2: Open VS Code

Open VS Code.

Open a terminal inside VS Code:

`Terminal → New Terminal`

or

`Ctrl + ``

---

## Step 3: Go to the folder where you want the project

Example:

```
cd Desktop
```

or

```
cd Documents
```

Check where you are:

```
pwd
```

---

## Step 4: Clone the repo

Run:

```
git clone <repo-link>
```

Example:

```
git clone https://github.com/username/brick-breaker-main.git
```

---

## If Git says "git is not recognized"

Run:

```
git --version
```

and tell me the output.

If Git isn't installed, we'll install it first.

---

## Step 5: Enter the project folder

After cloning:

```
cd brick-breaker-main
```

Check files:

```
dir
```

You should see something like:

```
main.py
requirements.txt
game/
```

---

## Step 6: Create a virtual environment (recommended)

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

You should see:

```
(venv)
```

at the start of the terminal line.

---

## Step 7: Install requirements

```
pip install -r requirements.txt
```

---

## Step 8: Run the game

```
python main.py
```

---

Send me:

1. The GitHub repo link (or screenshot)
2. Output of:

```
git --version
```

and we'll get it cloned properly.

**You said:**

git clone https://github.com/sanjanaaravamudhan/brick-breaker-main.git

PS C:\Users\Sanjan A> git clone https://github.com/sanjanaaravamudhan/brick-breaker-main.git

Cloning into 'brick-breaker-main'...

remote: Repository not found.

fatal: repository 'https://github.com/sanjanaaravamudhan/brick-breaker-main.git/' not found

**ChatGPT said:**

That error means one of three things:

## Possibility 1: Wrong URL

The repository might not actually be at:

`https://github.com/sanjanaaravamudhan/brick-breaker-main.git`

Even a small typo in the username or repo name will give:

`Repository not found`

---

## Possibility 2: Private Repository

If the repo is private, GitHub will often return:

`Repository not found`

when you're not authenticated or don't have access.

---

## Possibility 3: The repo hasn't been created yet

Sometimes assignment instructions provide the project structure but not the actual repository URL.

## Let's verify

Open this in your browser:

`https://github.com/sanjanaaravamudhan`

Do you see your GitHub profile?

Then check:

`https://github.com/sanjanaaravamudhan/brick-breaker-main`

Does it open, or does GitHub show a 404 page?

---

## Also tell me

Where did you get that URL from?

- GitHub Classroom?
- Moodle/Canvas/PES portal?
- A ZIP file download?
- A lab handout PDF?

A screenshot of the page where you got the repository link would help. Once I see that, I can tell whether we need to:

- clone a different URL,
- log into GitHub,
- accept a GitHub Classroom assignment first,
- or download a ZIP instead.

**You said:**

done i cloned

**ChatGPT said:**

Ayyy nice 😌

If the clone worked, move into the project folder:

```
cd brick-breaker-main
```

Check the contents:

```
dir
```

You should see something like:

```
main.py
requirements.txt
game/
README.md
```

Then install the dependencies:

```
pip install -r requirements.txt
```

After that, try running:

```
python main.py
```

and tell me:

1. Does the game window open?
2. Any errors in the terminal?
3. Paste the output of:

```
dir
```

from inside the project folder.

Also, for this lab, the very first thing I want to inspect is:

```
main.py
game/game_engine.py
```

So once the project is open in VS Code, either:

- paste those files here, or
- upload them.

Then we'll go through the code from absolute basics and map exactly where Tasks 1–4 need to be implemented.

**You said:**

alralr lets edit whatever code according to the requirements and lets update the code

**ChatGPT said:**

Absolutely, but I need the actual code first 🙂

Right now I know the **requirements**, but I don't know how your instructor implemented the game. There are a dozen different ways they could have written the collision logic, game loop, bricks, paddle, etc.

## Upload or paste these files:

`main.py`
`game/game_engine.py`
`game/ball.py`
`game/paddle.py`
`game/brick.py`

The fastest way in VS Code:

1. Open the file.
2. Ctrl+A
3. Ctrl+C
4. Paste it here.

Start with:

`game/game_engine.py`

because that's almost certainly where:

- collision detection lives (Task 1)
- win/loss detection lives (Task 2)
- replay logic will go (Task 3)
- sound triggering will go (Task 4)

---

Once I see the code, we'll do this properly:

## Task 1

✅ Fix side-vs-top brick collisions

## Task 2

✅ Add Pygame win/lose screen

### Task 3

✅ Difficulty selection + replay menu

### Task 4

✅ Sound effects

And I'll explain every change so you can justify it if the TA asks during evaluation.

Drop `game_engine.py` first and we'll start surgery. 🏓🧱

**You said:**

all of them are in binary though is that fine

**ChatGPT said:**

If by "binary" you mean VS Code is showing files like:

```
main.py
game_engine.py
ball.py
```

as normal Python text files, then that's **not** binary and you're good.

But if VS Code literally says something like:

`The file is not displayed in the text editor because it is either binary or uses an unsupported text encoding.`

then something's off.

A normal Brick Breaker assignment should contain readable `.py` files.

Can you tell me:

1. What file extensions do you see?

   ```
   .py
   .pyc
   .exe
   .pyd
   ```

   ?

2. In the terminal, run:

```
dir /s
```

or

```
Get-ChildItem -Recurse
```

and paste the output.

3. Or send a screenshot of the VS Code file explorer.

If the project only contains `.pyc` files (compiled Python bytecode), we'll handle it differently. If it contains normal `.py` files, we can open and edit them directly.

**You said:**

pyc

**ChatGPT said:**

Ahhh, if the project contains **`.pyc` files instead of `.py` files**, that's a very different situation.

`.pyc` files are **compiled Python bytecode**. They're meant to be run by Python, not edited by humans.

For example:

```
ball.pyc
brick.pyc
game_engine.pyc
```

instead of:

```
ball.py
brick.py
game_engine.py
```

means the source code may not have been included.

## First, let's confirm exactly what's in the repo

In PowerShell, run:

```
Get-ChildItem -Recurse
```

and paste the output here.

Or send a screenshot of the project tree in VS Code.

---

### If only `.pyc` files exist

We have a few options:

1. **Check if the `.py` files are hidden somewhere else in the repo.**
2. **Decompile the `.pyc` files** (often possible, depending on Python version).
3. **Rebuild the project from scratch** if it's small enough and the bytecode can't be cleanly decompiled.

Since the assignment expects you to modify collision detection, add menus, sounds, etc., it would be very unusual for an instructor to distribute only `.pyc` files. My first suspicion is that the actual source files are somewhere else in the repository.

Run:

```
tree /f
```

and send me the output. That will show the entire folder structure and we'll figure out what's really there.

---

If you want, I can:

- ↳ Share content of game_engine.py to start debugging collision issues
- ↳ List all files recursively in the project folder
- ↳ Upload or paste readable .py source files